

**LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK
KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)**

Disusun Oleh:

Muhammad Fharel

2511531010

Kelas Praktikum B

Tanggal Praktikum: 21 September 2026

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Ezza Addini



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

A. Langkah Kerja Praktikum

Praktikum Modul 3 ini bertujuan untuk memahami dan mengimplementasikan konsep Information Hiding serta Enkapsulasi dengan mengunci akses atribut menggunakan Access Modifiers (private). Selain itu, praktikum ini menambahkan mekanisme pengamanan transaksi menggunakan validasi PIN (Personal Identification Number) pada sistem perbankan.

1. Mengunci Entitas Transaksi (Transaksi.java)

Class Transaksi diubah agar seluruh atributnya bersifat private dan bersifat read-only (immutable) dengan hanya menyediakan method Getter tanpa Setter.

```
package pekan3_2511531010;

public class Transaksi {

    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public String getIdTransaksi() {
        return idTransaksi;
    }
    public String getJenis() {
        return jenis;
    }
    public double getNominal() {
        return nominal;
    }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
    }
}
```

Penjelasan Source Code Class Transaksi.java:

- Access Modifier private pada Atribut (idTransaksi, jenis, nominal)

Semua atribut dikunci menggunakan modifier private agar tidak dapat dimanipulasi secara langsung dari luar class.

- Implementasi Method Getter

Menyediakan method getIdTransaksi(), getJenis(), dan getNominal() untuk membaca nilai atribut secara aman tanpa memberikan akses untuk mengubahnya (Read-Only).

- Method Setter

Setter sengaja tidak dibuat karena objek transaksi dirancang bersifat immutable (tidak boleh diubah nilainya setelah dibuat).

2. Mengunci Entitas Rekening dan Menambahkan PIN (Rekening.java)

Class Rekening diperbarui dengan mengubah modifier atribut menjadi private, menambahkan atribut pin, memvalidasi panjang PIN di constructor, dan membuat method otentikasi internal.

```
package pekan3_2511531010;

import java.util.ArrayList;

public class Rekening {
    // 1. Mengunci atribut menjadi private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif!

    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
    }
}
```

```

// 3. Getter untuk atribut yang diizinkan dibaca publik
public String getNomorRekening() {
    return nomorRekening;
}
public String getNamaPemilik() {
    return namaPemilik;
}

// Method Otentikasi Internal
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal nominal
penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
    }
}

public void cetakMutasi() {
    System.out.println("== RIWAYAT MUTASI REKENING ==");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi t : riwayatTransaksi) {
            t.cetakDetail();
        }
    }
    System.out.println("=====");
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
}

```

Penjelasan Source Code Class Rekening.java:

- Atribut Private & Sensitif (pin)

Atribut `pin` ditambahkan untuk mengamankan hak akses akun. Penggunaan `private` menjamin PIN tidak bisa diakses langsung dari luar.

- Validasi PIN pada Constructor

Constructor memeriksa apakah panjang PIN bernilai tepat 6 digit (`pinAwal.length() == 6`). Jika tidak memenuhi kriteria, sistem memberikan peringatan dan mengatur PIN ke nilai default ("123456").

- Method otentikasi(`String inputPin`)

Method ini bertugas mencocokkan PIN masukan pengguna dengan PIN internal rekening. Mengembalikan nilai `true` jika cocok dan `false` jika salah.

Mengapa kita tidak membuat method `setSaldo()` dan `getPin()` ?

Kita tidak membuat `setSaldo()` supaya saldo tidak bisa diubah seenaknya dari luar program, karena saldo hanya boleh bertambah atau berkurang lewat proses setor dan tarik tunai yang resmi beserta catatannya. Kita juga tidak membuat `getPin()` untuk menjaga rahasia kata sandi pengguna, karena program lain cukup mengecek apakah PIN yang dimasukkan benar atau salah (lewat fungsi otentikasi) tanpa perlu tahu angka PIN aslinya.

B. Latihan Praktikum

Soal:

Sistem kita sekarang menuntut **PIN** saat pembuatan akun dan memiliki fungsi **otentikasi()**. Namun belum diimplementasikan pada menu interaksi di **Main.java**.

Tugas : Mengamankan CLI dengan Otentikasi

silahkan memperbarui program **Main.java** agar mematuhi aturan keamanan baru ini:

1. **Menu 1 (Buka Rekening):** Sebelum menginstansiasi objek baru, minta *user* menginputkan PIN (`String` angka 6 digit).
2. **Keamanan Transaksi:** Perbarui **Menu 3 (Tarik Tunai)** dan **Menu 6 (Cetak Mutasi)**. Sebelum sistem mengeksekusi kedua *method* tersebut, sistem harus meminta *user* memasukkan PIN.
 - Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
 - Jika mengembalikan nilai `true`, jalankan transaksi.
 - Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Kode Program:

1. Menu 1 (Buka Rekening):

```
27 case 1:
28     System.out.print("Masukkan No Rekening: ");
29     String no = scanner.nextLine();
30     System.out.print("Masukkan Nama Pemilik: ");
31     String nama = scanner.nextLine();
32     System.out.print("Masukkan Saldo Awal: ");
33     double saldoAwal = scanner.nextDouble();
34     scanner.nextLine();
35     System.out.print("Masukkan PIN (6 digit): ");
36     String pin = scanner.nextLine();
37
38     akunAktif = new Rekening(no, nama, saldoAwal, pin);
39     daftarRekening.add(akunAktif);
40     break;
```

Penjelasan kode tugas 1:

- Program menerima input nomor rekening, nama pemilik, saldo awal, dan variabel pin berformat String 6 digit dari pengguna.
- Nilai pin diteruskan ke dalam konstruktor Rekening bersama data rekening lainnya.
- Di dalam konstruktor Rekening, variabel pin divalidasi dengan logika if (pinAwal.length() == 6). Jika panjang PIN 6 digit, data disimpan ke atribut private pin. Jika kurang atau lebih dari 6 digit, sistem memunculkan peringatan dan secara otomatis menetapkan PIN default "123456".
- Akun baru yang berhasil dibuat ditambahkan ke daftarRekening dan diset sebagai akunAktif.

2. Menu 3 dan 6 (Keamanan Transaksi)

```
52 case 3:
53     if (akunAktif == null) {
54         System.out.println("Belum ada rekening aktif! Silahkan buka rekening terlebih dahulu.");
55         break;
56     }
57     System.out.print("Masukkan PIN Anda: ");
58     String pinTarik = scanner.nextLine();
59
60     if (akunAktif.otentikasi(pinTarik)) {
61         System.out.print("Masukkan nominal tarik: ");
62         double nominalTarik = scanner.nextDouble();
63         akunAktif.tarikTunai(nominalTarik);
64     } else {
65         System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
66     }
67     break;
```

Penjelasan kode tugas 2 (Menu 3):

- Pengecekan Keberadaan Akun:
Memastikan variabel akunAktif tidak bernilai null.
- Input PIN Penarikan:
Meminta pengguna memasukkan PIN sebelum melangkah ke proses penarikan.
- Eksekusi Otentikasi:
Memanggil akunAktif.otentikasi(pinTarik). Method ini membandingkan pinTarik dengan atribut private pin pada kelas Rekening menggunakan this.pin.equals(inputPin).
- Percabangan Hasil:
Jika bernilai true, program meminta input nominal tarik dan menjalankan method akunAktif.tarikTunai(nominalTarik).
Jika bernilai false, penarikan dibatalkan dan sistem mencetak pesan kesalahan: "Akses Ditolak: PIN yang Anda masukkan salah!".

```

100 case 6:
101     if (akunAktif == null) {
102         System.out.println("Error: Anda belum membuka rekening! Silahkan buka rekening terlebih dahulu.");
103     } else {
104         System.out.print("Masukkan PIN: ");
105         String pinMutasi = scanner.nextLine();
106
107         if (akunAktif.otentikasi(pinMutasi)) {
108             akunAktif.cetakMutasi();
109         } else {
110             System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
111         }
112     }
113     break;

```

Penjelasan kode tugas 2 (Menu 6):

- Pengecekan Keberadaan Akun:
Memastikan ada akun yang sedang aktif.
- Otentikasi Akses Mutasi:
Meminta masukan pinMutasi untuk melindungi kerahasiaan riwayat transaksi.
- Eksekusi Otentikasi:
Mengirimkan pinMutasi ke method akunAktif.otentikasi(pinMutasi).
- Percabangan Hasil:

Jika bernilai true, method `akunAktif.cetakMutasi()` dipanggil untuk menampilkan seluruh riwayat transaksi.

Jika bernilai false, sistem membatalkan pencetakan mutasi dan menampilkan pesan: "Akses Ditolak: PIN yang Anda masukkan salah!".

Contoh output Menu 1:

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456789
Masukkan Nama Pemilik: Jennie
Masukkan Saldo Awal: 300000
Masukkan PIN (6 digit): 123456
Rekening atas nama Jennie berhasil dibuat.
```

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 098765
Masukkan Nama Pemilik: Jennie
Masukkan Saldo Awal: 300000
Masukkan PIN (6 digit): 1234
Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456
Rekening atas nama Jennie berhasil dibuat.
```

Output Menu 1 (Buka Rekening Baru) menunjukkan dua skenario pembuatan akun pada Sistem Perbankan Mini. Pada skenario pertama, pengguna memilih opsi menu 1 lalu memasukkan nomor rekening, nama pemilik, saldo awal, dan PIN 6 digit (123456) yang valid, sehingga akun atas nama Jennie berhasil dibuat secara normal. Sementara itu, pada

skenario kedua, pengguna memasukkan PIN yang tidak valid karena kurang dari 6 digit (1234). Sistem secara otomatis mendeteksi ketidaksesuaian tersebut, memberikan pesan peringatan, dan menetapkan PIN default 123456 untuk menjaga integritas data sebelum akhirnya akun berhasil dibuat.

Contoh output Menu 3:

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan PIN Anda: 123456
Masukkan nominal tarik: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp250000.0

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan PIN Anda: 999999
Akses Ditolak: PIN yang Anda masukkan salah!
```

Output Menu 3 (Tarik Tunai) menampilkan dua skenario otentikasi keamanan saat pengguna melakukan penarikan uang pada Sistem Perbankan Mini. Pada skenario pertama, pengguna memilih opsi menu 3 dan memasukkan PIN yang benar (123456), sehingga sistem memberikan izin untuk memasukkan nominal penarikan (50000) dan berhasil memperbarui sisa saldo menjadi Rp250.000,0. Sebaliknya, pada skenario kedua, pengguna memasukkan PIN yang salah (999999), yang membuat sistem langsung memblokir akses transaksi dan menampilkan pesan kegagalan "Akses Ditolak: PIN yang Anda masukkan salah!" demi menjaga keamanan rekening.

Contoh output Menu 6:

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Masukkan PIN: 123456
=== RIWAYAT MUTASI REKENING ===
ID: TRX-T-1790527392041 | Jenis: Debit | Nominal: Rp50000.0
=====

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Masukkan PIN: 99999
Akses Ditolak: PIN yang Anda masukkan salah!
```

Output Menu 6 (Cetak Mutasi) menampilkan dua skenario otentikasi PIN saat pengguna ingin melihat riwayat transaksi pada Sistem Perbankan Mini. Pada skenario pertama, pengguna memilih menu 6 dan memasukkan PIN yang sesuai (123456), sehingga sistem berhasil menampilkan riwayat mutasi rekening yang memuat ID transaksi, jenis transaksi (Debit), dan nominalnya. Sebaliknya, pada skenario kedua, pengguna memasukkan PIN yang tidak valid (9999), sehingga sistem menolak akses informasi mutasi dan menampilkan pesan penolakan "Akses Ditolak: PIN yang Anda masukkan salah!" guna menjaga kerahasiaan data transaksi nasabah.

C. Link Repository GitHub

https://github.com/MuhammadFharel/PrakPBO_B_2511531010